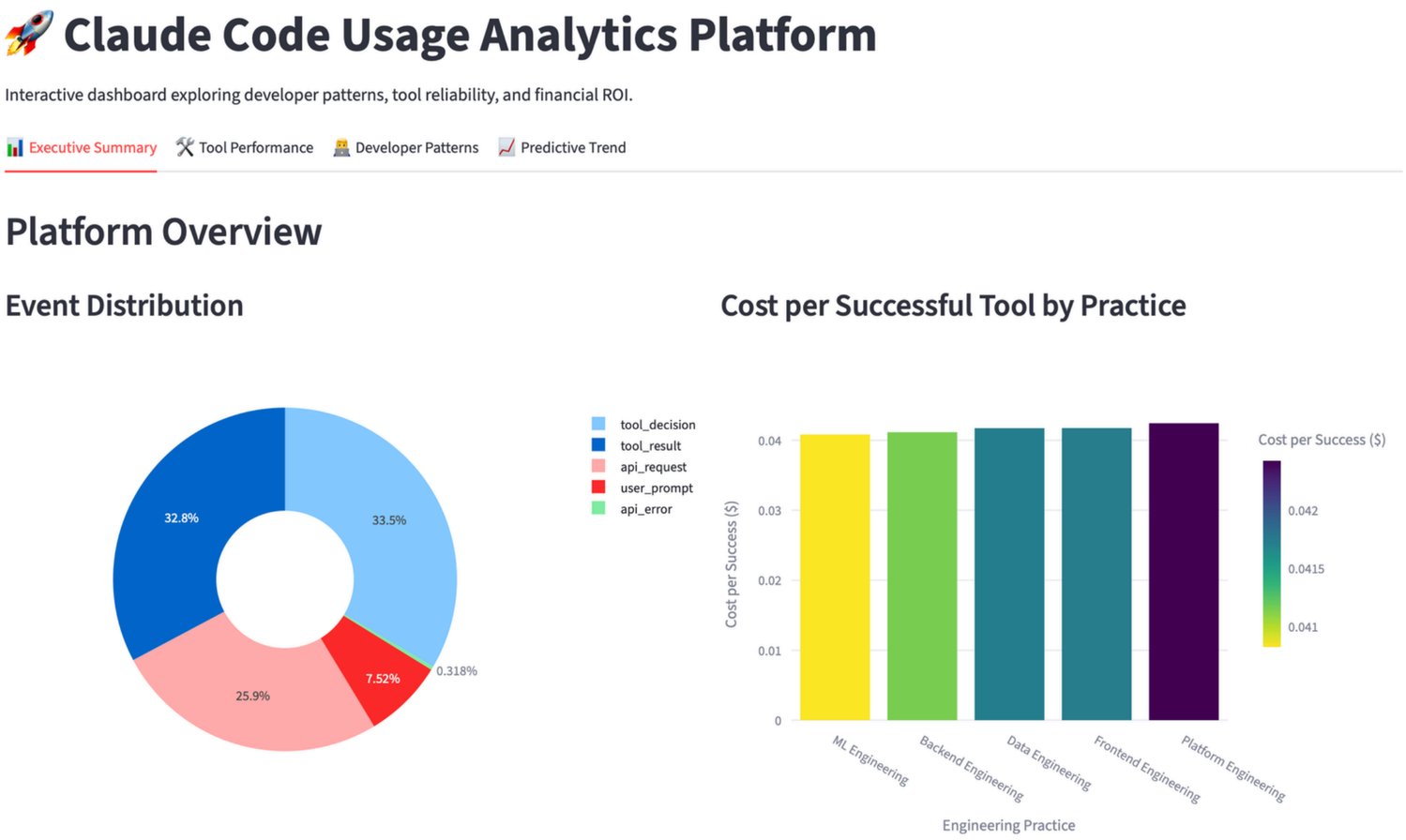
Claude Code Usage Analytics

Milena Bazoyan

13 March, 2026

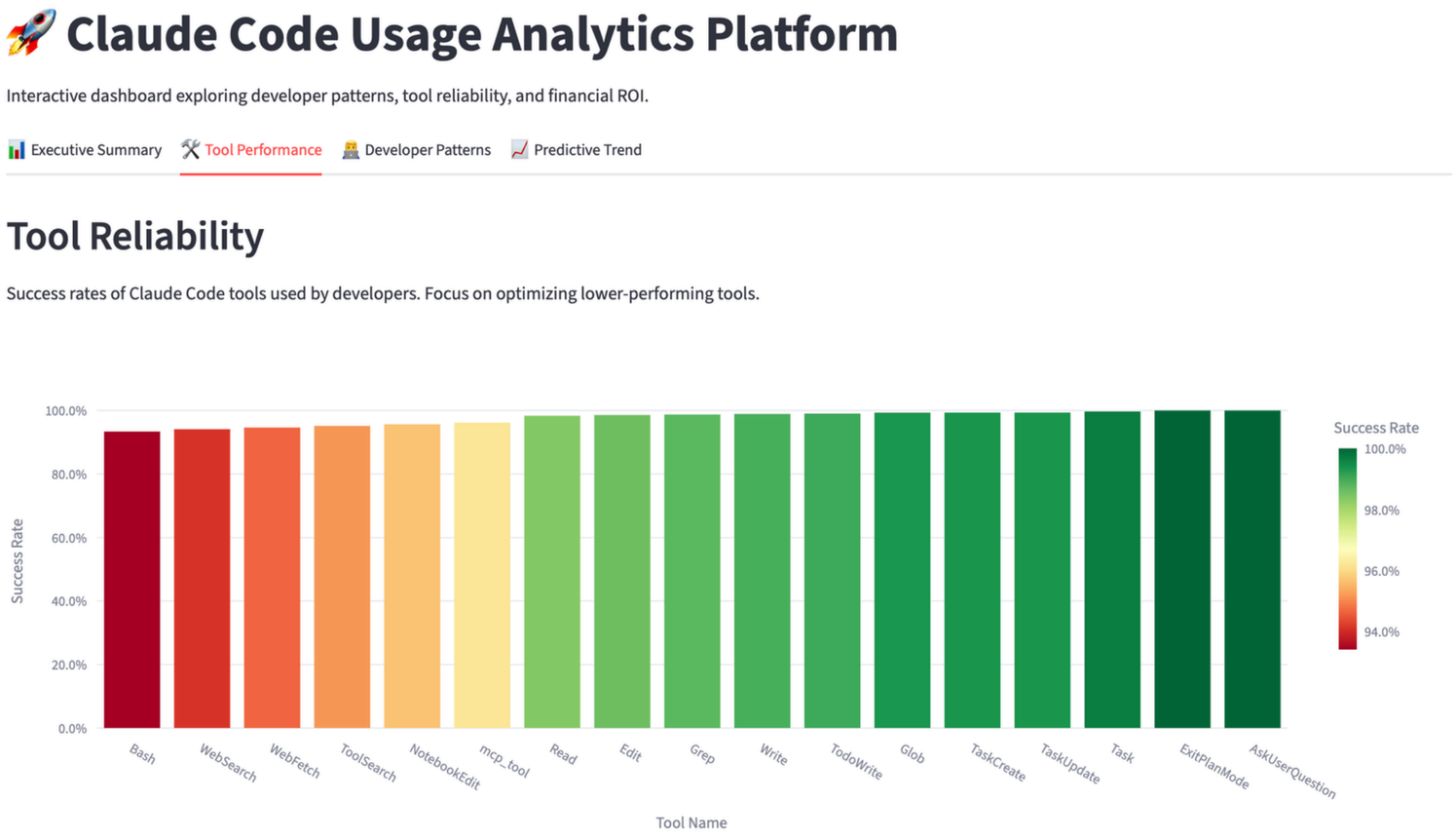
EXECUTIVE SUMMARY & PLATFORM OVERVIEW

- Engineered a Python/Pandas pipeline to ingest, clean, and merge raw JSONL telemetry streams with employee metadata.
- Built an interactive *Streamlit dashboard* to transform raw data into actionable stakeholder insights.
- **Event Distribution (Chart 1):** A pie chart breaking down platform activity, showing the ratio of user prompts to API requests and tool actions.
- **Practice ROI (Chart 2):** A bar chart measuring cost-efficiency by dividing a department's total API spend by their number of successful tool executions.
- **Key Insight**: Volume does not equal efficiency. *ML Engineering is our most cost-efficient team* (~$0.04/success), while *Platform Engineering requires the most API* spend per successful action.



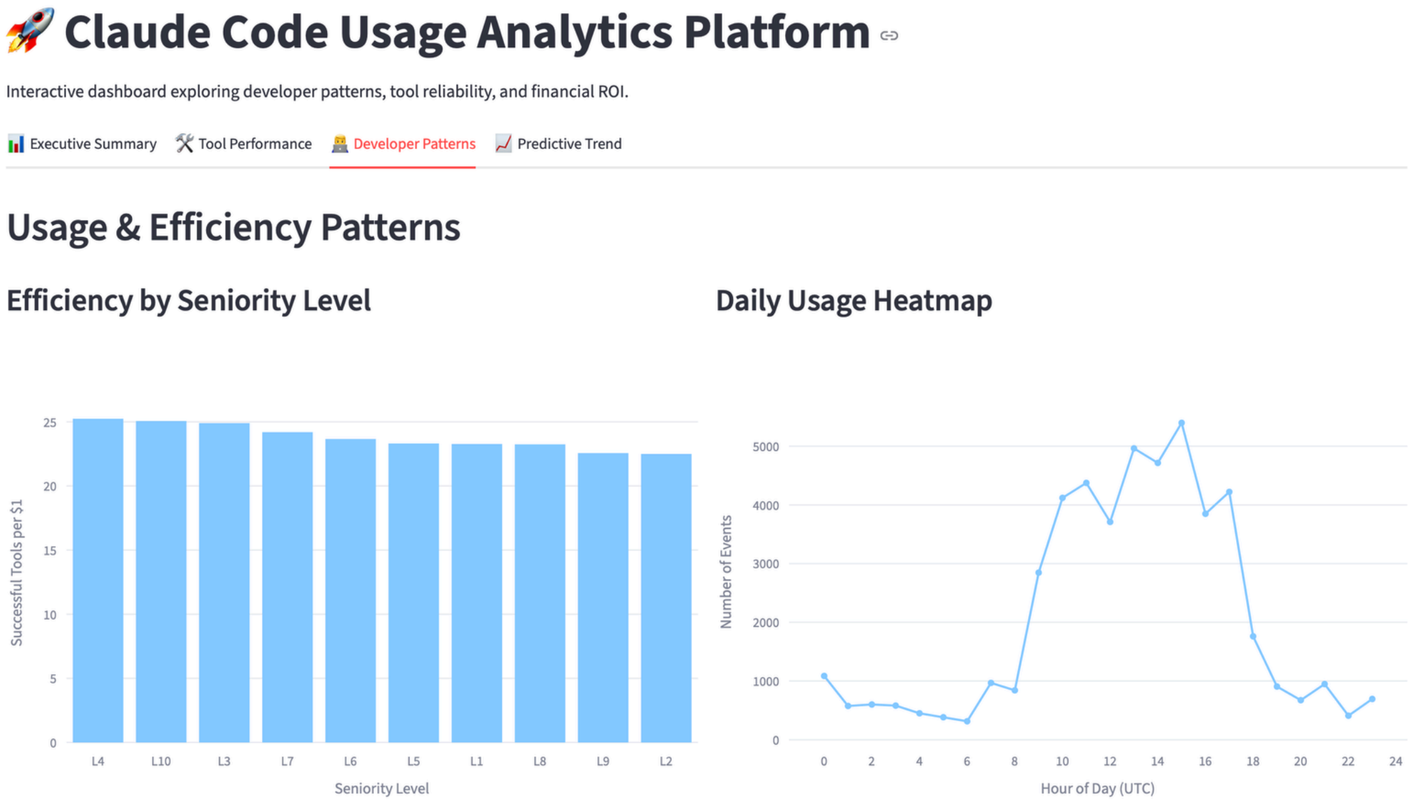
EXECUTIVE SUMMARY & PLATFORM OVERVIEW

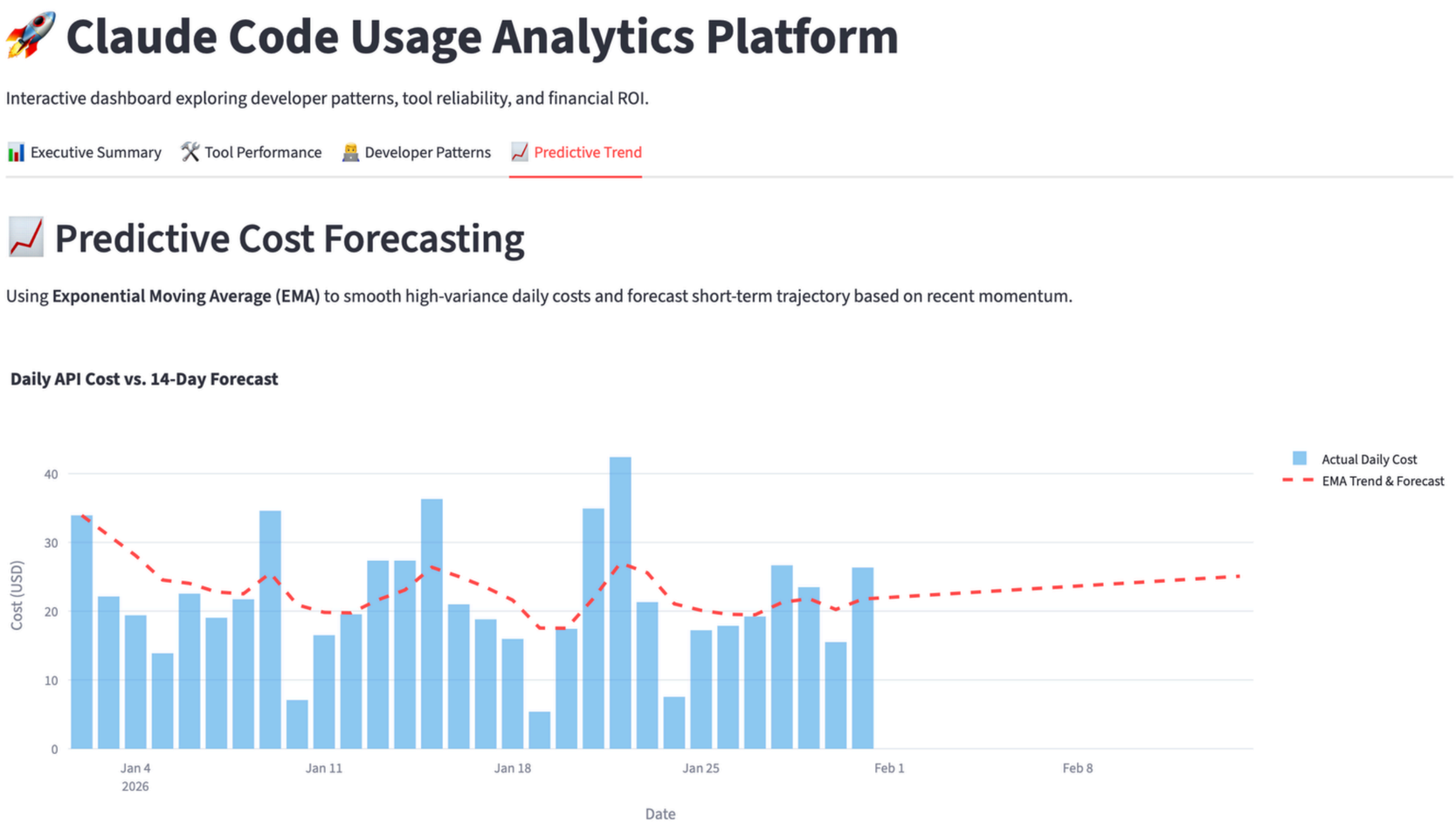
- Evaluated the success versus error rates of specific Claude Code tools (e.g., Bash, Read, Edit) to identify where developers experience bottlenecks.
- **The Chart**: A bar chart ranking each tool by its overall success rate percentage across all analyzed sessions.
- **Key Insight:** Simple informational tools (like Read) boast near-perfect reliability, while complex execution tools (like Bash) show higher failure rates (~93%).
- **Actionable Takeaway**: We should provide developers with advanced prompt-engineering guidelines specifically for guiding Claude through complex, multi-step Bash commands to reduce expensive API retries.



EXECUTIVE SUMMARY & PLATFORM OVERVIEW

- Mapped user efficiency against developer seniority levels and tracked hourly API usage to find when developers rely on the AI most.
- **Key Insight 1 (The Seniority Paradox):** Mid-level engineers (L4) extract the most value per dollar spent (~25 successes per $1), significantly outperforming both juniors and highly senior staff.
- **Key Insight 2 (Peak Usage):** The temporal heatmap reveals concentrated spikes in API activity during core engineering hours, showing exactly when the team tackles their hardest coding blocks.
- **Actionable Takeaway:** We should optimize API budget and infrastructure scaling around these daily peak hours, and study L4 developer workflows to create best-practice guides for the rest of the company.



EXECUTIVE SUMMARY & PLATFORM OVERVIEW

- Implemented a machine learning component **(Exponential Moving Average)** to model historical API usage and predict short-term future expenditures.
- **The Chart:** A combined visualization overlaying high-variance, day-to-day API costs (bars) with a smoothed, 14-day forward-looking trendline.
- **Key Insight:** The EMA model *successfully* filters out *random daily usage spikes* to reveal the true underlying momentum and trajectory of our API adoption.
- **Actionable Takeaway:** Leadership can use this 14-day projection to proactively manage budget allocations and monitor for unexpected spending surges.